

Competitive Programming

Section 37 Editorial: Segment Trees

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
2	A. Range Minimum Updates	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Maximum Subarray Updates	7
3.1	Restatement	7
3.2	Algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	8
4	C. Lazy Range Add Sum	10
4.1	Restatement	10
4.2	Algorithm	10
4.3	Correctness proof	10
4.4	Complexity	10
4.5	Full C++ solution	10
4.6	Full Python solution	12
5	D. First Available	14
5.1	Restatement and constraint implications	14
5.2	Node invariant and trace	14
5.3	Algorithm	14
5.4	Correctness proof	15
5.5	Complexity, implementation notes, and tests	15
5.6	Full C++ solution	15
5.7	Full Python solution	16
6	Testing discussion	18
7	Problem-specific upsolve notes	19
7.1	A. Range Minimum Updates	19
7.2	B. Maximum Subarray Updates	19
7.3	C. Lazy Range Add Sum	20
7.4	D. First Available	20
8	Stress-testing workflow	21

1 Editorial overview

Problem	Main idea
A. Range Minimum Updates	See full algorithm and code below.
B. Maximum Subarray Updates	See full algorithm and code below.
C. Lazy Range Add Sum	See full algorithm and code below.
D. First Available	Store interval maxima, then descend to the leftmost feasible leaf.

2 A. Range Minimum Updates

2.1 Restatement

Point-assign values and answer range minimum queries.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
int size = 1;
while (size < n) size <= 1;
const long long INF = (1LL << 60);
vector<long long> seg(2 * size, INF);
for (int i = 0; i < n; i++) cin >> seg[size + i];
for (int i = size - 1; i >= 1; i--) seg[i] = min(seg[2 * i], seg[2 * i + 1]);
while (q--) {
    int typ, x, y;
    cin >> typ >> x >> y;
    if (typ == 1) {
        int p = size + x - 1;
        seg[p] = y;
        for (p >= 1; p; p >= 1) seg[p] = min(seg[2 * p], seg[2 * p + 1]);
    } else {
        int l = size + x - 1, r = size + y;
        long long ans = INF;
        while (l < r) {
            if (l & 1) ans = min(ans, seg[l++]);
            if (r & 1) ans = min(ans, seg[--r]);
            l >>= 1;
            r >>= 1;
        }
        cout << ans << '\n';
    }
}
return 0;
}

```

2.6 Full Python solution

```

import sys

INF = 10**30

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    size = 1
    while size < n:
        size *= 2
    seg = [INF] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = x
    for i in range(size - 1, 0, -1):

```

```

        seg[i] = min(seg[2 * i], seg[2 * i + 1])
    idx = 2 + n
    out = []
    for _ in range(q):
        typ, x, y = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        if typ == 1:
            p = size + x - 1
            seg[p] = y
            p //= 2
            while p:
                seg[p] = min(seg[2 * p], seg[2 * p + 1])
                p //= 2
        else:
            l, r = x - 1 + size, y + size
            ans = INF
            while l < r:
                if l & 1:
                    ans = min(ans, seg[l])
                    l += 1
                if r & 1:
                    r -= 1
                    ans = min(ans, seg[r])
                l //= 2
                r //= 2
            out.append(str(ans))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 B. Maximum Subarray Updates

3.1 Restatement

After each point assignment, print the maximum non-empty subarray sum of the whole array.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long NEG = -(1LL << 60);

struct Node {
    long long sum, pref, suff, best;
```

```

};

Node leaf(long long x) { return {x, x, x, x}; }
Node merge(Node a, Node b) {
    return {
        a.sum + b.sum,
        max(a.pref, a.sum + b.pref),
        max(b.suff, b.sum + a.suff),
        max({a.best, b.best, a.suff + b.pref})
    };
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    int size = 1;
    while (size < n) size <= 1;
    Node neutral{0, NEG, NEG, NEG};
    vector<Node> seg(2 * size, neutral);
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        seg[size + i] = leaf(x);
    }
    for (int i = size - 1; i >= 1; i--) seg[i] = merge(seg[2 * i], seg[2 * i + 1]);
    while (q--) {
        int pos;
        long long val;
        cin >> pos >> val;
        int p = size + pos - 1;
        seg[p] = leaf(val);
        for (p >= 1; p; p >= 1) seg[p] = merge(seg[2 * p], seg[2 * p + 1]);
        cout << seg[1].best << '\n';
    }
    return 0;
}

```

3.6 Full Python solution

```

import sys

NEG = -10**30

def merge(a, b):
    total = a[0] + b[0]
    pref = max(a[1], a[0] + b[1])

```

```

    suff = max(b[2], b[0] + a[2])
    best = max(a[3], b[3], a[2] + b[1])
    return (total, pref, suff, best)

def leaf(x):
    return (x, x, x, x)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    size = 1
    while size < n:
        size *= 2
    neutral = (0, NEG, NEG, NEG)
    seg = [neutral] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = leaf(x)
    for i in range(size - 1, 0, -1):
        seg[i] = merge(seg[2 * i], seg[2 * i + 1])
    idx = 2 + n
    out = []
    for _ in range(q):
        pos, val = data[idx], data[idx + 1]
        idx += 2
        p = size + pos - 1
        seg[p] = leaf(val)
        p //= 2
        while p:
            seg[p] = merge(seg[2 * p], seg[2 * p + 1])
            p //= 2
        out.append(str(seg[1][3]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

4 C. Lazy Range Add Sum

4.1 Restatement

Support range add updates and range sum queries.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct SegTree {
    int n;
    vector<long long> seg, lazy;
    SegTree(const vector<long long>& a) {
```

```

    n = 1;
    while (n < (int)a.size()) n <= 1;
    seg.assign(2 * n, 0);
    lazy.assign(2 * n, 0);
    for (int i = 0; i < (int)a.size(); i++) seg[n + i] = a[i];
    for (int i = n - 1; i >= 1; i--) seg[i] = seg[2 * i] + seg[2 * i + 1];
}

void apply(int v, int len, long long x) {
    seg[v] += x * len;
    lazy[v] += x;
}

void push(int v, int len) {
    if (lazy[v] && v < n) {
        int half = len / 2;
        apply(2 * v, half, lazy[v]);
        apply(2 * v + 1, half, lazy[v]);
        lazy[v] = 0;
    }
}

void add(int v, int tl, int tr, int l, int r, long long x) {
    if (l <= tl && tr <= r) {
        apply(v, tr - tl + 1, x);
        return;
    }
    push(v, tr - tl + 1);
    int tm = (tl + tr) / 2;
    if (l <= tm) add(2 * v, tl, tm, l, r, x);
    if (tm < r) add(2 * v + 1, tm + 1, tr, l, r, x);
    seg[v] = seg[2 * v] + seg[2 * v + 1];
}

long long query(int v, int tl, int tr, int l, int r) {
    if (l <= tl && tr <= r) return seg[v];
    push(v, tr - tl + 1);
    int tm = (tl + tr) / 2;
    long long ans = 0;
    if (l <= tm) ans += query(2 * v, tl, tm, l, r);
    if (tm < r) ans += query(2 * v + 1, tm + 1, tr, l, r);
    return ans;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<long long> a(n);
    for (long long &x : a) cin >> x;
    SegTree st(a);
    while (q--) {
        int typ, l, r;

```

```

    cin >> typ >> l >> r;
    if (typ == 1) {
        long long x;
        cin >> x;
        st.add(1, 1, st.n, l, r, x);
    } else {
        cout << st.query(1, 1, st.n, l, r) << '\n';
    }
}
return 0;
}

```

4.6 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    size = 1
    while size < n:
        size *= 2
    seg = [0] * (2 * size)
    lazy = [0] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = x
    for i in range(size - 1, 0, -1):
        seg[i] = seg[2 * i] + seg[2 * i + 1]
    def apply(v, length, delta):
        seg[v] += delta * length
        lazy[v] += delta
    def push(v, length):
        if lazy[v] and v < size:
            half = length // 2
            apply(2 * v, half, lazy[v])
            apply(2 * v + 1, half, lazy[v])
            lazy[v] = 0
    def add(v, tl, tr, l, r, x):
        if l <= tl and tr <= r:
            apply(v, tr - tl + 1, x)
            return
        push(v, tr - tl + 1)
        tm = (tl + tr) // 2
        if l <= tm:
            add(2 * v, tl, tm, l, r, x)
        if tm < r:

```

```

        add(2 * v + 1, tm + 1, tr, l, r, x)
    seg[v] = seg[2 * v] + seg[2 * v + 1]
def query(v, tl, tr, l, r):
    if l <= tl and tr <= r:
        return seg[v]
    push(v, tr - tl + 1)
    tm = (tl + tr) // 2
    ans = 0
    if l <= tm:
        ans += query(2 * v, tl, tm, l, r)
    if tm < r:
        ans += query(2 * v + 1, tm + 1, tr, l, r)
    return ans
idx = 2 + n
out = []
for _ in range(q):
    typ = data[idx]
    idx += 1
    if typ == 1:
        l, r, x = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        add(1, 1, size, l, r, x)
    else:
        l, r = data[idx], data[idx + 1]
        idx += 2
    out.append(str(query(1, 1, size, l, r)))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

5 D. First Available

5.1 Restatement and constraint implications

There are n desks, each with a non-negative capacity. A type 1 $i \ x$ operation replaces desk i 's capacity by x . A type 2 x operation must print the *smallest* desk index whose current capacity is at least x , or 0 if no such desk exists. There are up to 200,000 desks and operations, and capacities can be as large as 10^9 .

A direct scan from the first desk is easy to describe, but one query can then cost $O(n)$. In a sequence dominated by type-2 requests this becomes quadratic. We need a summary that can discard an entire interval as soon as it cannot contain a feasible desk.

5.2 Node invariant and trace

Store the **maximum capacity** in every tree node's interval. The invariant is simple: `tree[v]` equals the maximum of all current capacities represented by node v . A node whose maximum is smaller than x contains no valid answer; a node whose maximum is at least x contains at least one valid answer.

For capacities $[3, 0, 4, 2, 1]$, a request for $x=3$ starts at a root with maximum 4. Its left child covers $[1,4]$ and has maximum 4, so the answer is in the left child. The next left child covers $[1,2]$ and has maximum 3, so continue left. At the final split, position 1 has capacity 3, so it is the answer. A request for $x=4$ first rejects $[1,2]$ because its maximum is 3, then reaches position 3.

The important ordering rule is: inspect the left child first. Going right merely because both children are feasible would return some valid desk, but not necessarily the smallest one.

5.3 Algorithm

Build a power-of-two iterative segment tree using maximum as the merge. Pad leaves beyond n with -1 ; all real capacities are non-negative and every request has $x \geq 1$, so padding cannot be selected.

```
assign(i, x):
    replace leaf i by x
    recompute maxima on the path to the root

first_at_least(x):
    if root maximum < x:
        return 0
    v = root
    while v is not a leaf:
```

```

    if maximum(left child of v) >= x:
        v = left child
    else:
        v = right child
    return index represented by v

```

5.4 Correctness proof

We first prove the node invariant. It is true for a leaf because its stored value is exactly that desk capacity. Every internal node is recomputed as the maximum of its two children, so if both child invariants are true, it equals the maximum capacity in their combined interval. Building bottom-up and recomputing every ancestor after an assignment therefore preserve the invariant.

For a request x , if the root maximum is below x , the invariant says every desk is below x , so returning zero is correct. Otherwise, at every internal node reached by the descent, its interval contains a qualifying desk. If the left child maximum is at least x , its interval contains one and every index there is smaller than every index in the right child, so descending left keeps the smallest possible answer. If it is below x , the left interval has no qualifying desk and the right interval must contain one, so descending right also keeps the smallest possible answer. At a leaf, that leaf is exactly the leftmost qualifying desk. Thus every printed answer is correct. QED.

5.5 Complexity, implementation notes, and tests

Building takes $O(n)$. Both an assignment and a search visit one root-to-leaf path, so they take $O(\log n)$ time. The tree uses $O(n)$ memory. Use `long long` in C++ for capacities even though the stated bound fits in an `int`; this makes the maximum state robust if the statement is later extended.

Test all capacities below a request, multiple equal feasible capacities, assignments that create a new earlier answer, and $n=1$. Do not use zero as padding: a request for zero would then select a fake leaf. The local contract uses requests at least one, but -1 padding makes the intent unambiguous.

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    int size = 1;
    while (size < n) size <= 1;

```

```

vector<long long> tree(2 * size, -1);
for (int i = 0; i < n; i++) cin >> tree[size + i];
for (int v = size - 1; v >= 1; v--) {
    tree[v] = max(tree[2 * v], tree[2 * v + 1]);
}

while (q--) {
    int type;
    cin >> type;
    if (type == 1) {
        int index;
        long long capacity;
        cin >> index >> capacity;
        int v = size + index - 1;
        tree[v] = capacity;
        for (v >= 1; v > 0; v >= 1) {
            tree[v] = max(tree[2 * v], tree[2 * v + 1]);
        }
    } else {
        long long need;
        cin >> need;
        if (tree[1] < need) {
            cout << 0 << '\n';
            continue;
        }
        int v = 1;
        while (v < size) {
            if (tree[2 * v] >= need) v = 2 * v;
            else v = 2 * v + 1;
        }
        cout << v - size + 1 << '\n';
    }
}
return 0;
}

```

5.7 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    size = 1
    while size < n:

```

```

size *= 2

tree = [-1] * (2 * size)
tree[size : size + n] = data[2 : 2 + n]
for node in range(size - 1, 0, -1):
    tree[node] = max(tree[2 * node], tree[2 * node + 1])

pointer = 2 + n
output = []
for _ in range(q):
    operation = data[pointer]
    pointer += 1
    if operation == 1:
        index, capacity = data[pointer], data[pointer + 1]
        pointer += 2
        node = size + index - 1
        tree[node] = capacity
        node //= 2
        while node:
            tree[node] = max(tree[2 * node], tree[2 * node + 1])
            node //= 2
    else:
        need = data[pointer]
        pointer += 1
        if tree[1] < need:
            output.append("0")
            continue
        node = 1
        while node < size:
            if tree[2 * node] >= need:
                node *= 2
            else:
                node = 2 * node + 1
        output.append(str(node - size + 1))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

6 Testing discussion

The fixed samples cover visible behavior. The original random tests exercise many short operation sequences. D additionally keeps a literal-array oracle: it scans from the first desk on tiny cases instead of invoking either reference implementation, then adds a 200,000-operation scale case with a directly known answer. The resulting expected files are used against both the C++ and Python submissions.

7 Problem-specific upsolve notes

7.1 A. Range Minimum Updates

Constraint implication and trap. Point assignments mean a sparse table is invalid after the first update; repeated direct range scans are too slow. Use `min` as the merge and `INF` as the padding identity for leaves beyond `n`.

Trace and pseudocode. In `[5,4,3,2,1]`, assign position 3 to 10. Rebuild only its ancestors; query `[2,4]` returns `min(4,10,2)=2`.

```
assign(p,x): set leaf p=x; recompute ancestors with min
query(l,r): merge the disjoint canonical nodes covering [l,r]
```

By induction from leaves, every node stores the minimum of its segment. The query nodes partition the requested interval, so their minimum is correct. Build is $O(n)$, each assignment/query is $O(\log n)$, memory $O(n)$. C++ uses `long long INF`; Python's `10**30` must exceed all inputs. Test singleton queries, updates at both ends, and all-positive values (which expose zero padding).

7.2 B. Maximum Subarray Updates

Observation. A node needs `(sum,pref,suff,best)`, all for non-empty subarrays. On merging `L` then `R`, a crossing answer is only `L.suff+R.pref`. For `[1,-2]` merged with `[3,4]`, candidates are left best 1, right best 7, crossing 6; the answer is 7.

```
merge(L,R):
    sum=L.sum+R.sum
    pref=max(L.pref,L.sum+R.pref)
    suff=max(R.suff,R.sum+L.suff)
    best=max(L.best,R.best,L.suff+R.pref)
```

Every non-empty subarray of the union lies fully left, fully right, or crosses the boundary, proving the formulas and root answer. A point assignment rebuilds one root path: $O(\log n)$, $O(n)$ memory. Use negative infinity in neutral padding, not zero. Test all-negative arrays, replacing the current best, and `n=1`.

7.3 C. Lazy Range Add Sum

Invariant. `seg[v]` already includes every update covering its entire interval; `lazy[v]` is the delta still owed to children. `apply(v,len,x)` adds $x \cdot \text{len}$ to its sum and adds x to its tag. On partial overlap, push before descending, recurse into intersecting children, then recompute the parent.

For `[1,2,3,4]`, adding 3 to `[1,4]` changes the root sum by 12 without visiting leaves. Query `[2,3]` first pushes the tag, then returns $5+6=11$. The invariant proves a fully covered node answers immediately, while pushing makes partial descendants correct. Build $O(n)$; update/query $O(\log n)$; memory $O(n)$. Test a full update followed by partial query, overlapping updates, negative deltas, and an update ending at `n`.

7.4 D. First Available

Invariant. A node stores the maximum capacity in its interval. A request for `x` can discard any node whose maximum is below `x`; if the root survives, always inspect the left child before the right child. For `[3,0,4,2,1]`, a request for 4 rejects the `[1,2]` interval and reaches index 3. This left-first descent is what turns an ordinary maximum tree into a leftmost-answer search.

```
if tree[1] < x: answer = 0
otherwise descend left when left.max >= x, else descend right
```

The maximum invariant proves rejection is safe and the left-first choice proves the leaf is the smallest feasible index. Both operations are $O(\log n)$. Test no feasible desk, ties, a new earlier capacity after an assignment, and the one-desk case. A literal array scan is an effective small random oracle.

8 Stress-testing workflow

Compare each operation to a literal list on short random cases. For B, recompute the answer with Kadane's algorithm after every assignment. Then run `CP_TARGET=solution python3 sections/37_segment_trees/check.py`. Carry forward: design the node state from the merge cases, and state the lazy-tag meaning before writing recursion.